Product Chatbot with Azure AI Foundry + RAG

A step-by-step learning guide: build it locally, deploy it to Azure App Service, and secure it with Microsoft Entra ID

October 2026

Contents

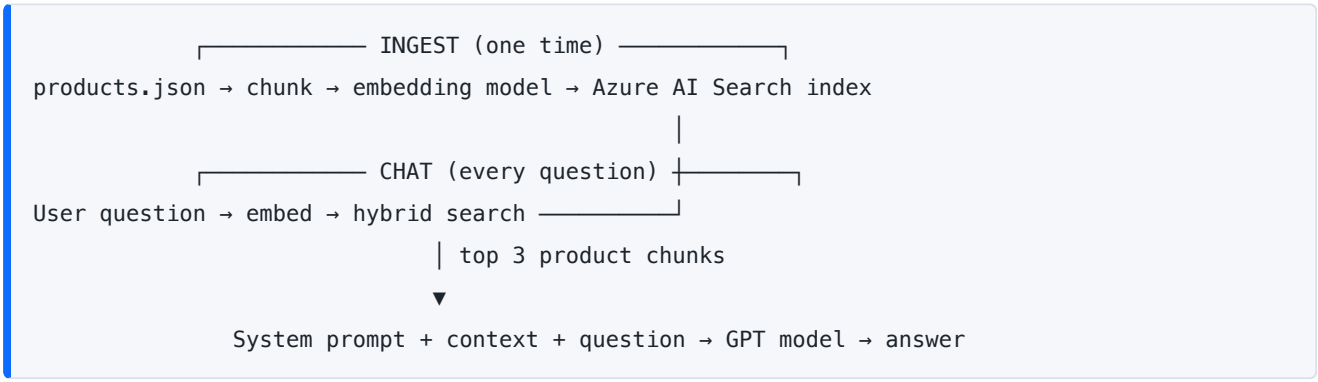
- **Part 1: Build the RAG chatbot (command line)**: Foundry project, model deployments, Azure AI Search, ingestion, and a chat loop
- **Part 2: Turn it into a web app on Azure App Service**: Flask API, chat UI, deployment, troubleshooting, and cleanup
- **Part 3: Add Microsoft Entra ID authentication**: keyless access with Managed Identity, user sign-in with Easy Auth, and the final cleanup

> **How to use this guide:** Follow the parts in order. Part 1 starts with API keys because that's the easiest way to learn. Part 3 then removes the keys and replaces them with Entra ID. If you already know you'll finish Part 3, you can still type the Part 1 and Part 2 code as written, because Part 3 shows exactly what to change.
> **Note:** Microsoft renamed "Azure AI Studio" to "Azure AI Foundry", and the portal now also uses the name "Microsoft Foundry". Some menu labels may differ slightly from what's written here. The concepts stay the same.

Part 1: Build the RAG chatbot

This part builds the RAG pipeline in code so you can see each part: chunking, embeddings, vector search, and grounding.

Architecture



Step 0: Prerequisites

- An Azure subscription. The free trial works.
- Python 3.10 or newer.
- Azure CLI: `brew install azure-cli`

Step 1: Create a Foundry project

1. Go to https://ai.azure.com and sign in.
2. Click **Create project**.
3. Name it `product-chatbot`. Let it create a new **resource group**, e.g. `rg-product-chatbot`.
4. Pick a region that has GPT-4o-mini and embedding models available, e.g. **East US 2** or **Sweden Central**.
5. When it finishes, open the project **Overview** page and copy:
    - the **Azure OpenAI endpoint** (looks like `https://<name>.openai.azure.com/`)
    - the **API key**

Step 2: Deploy two models

In your project, go to **Models + endpoints** → **Deploy model** → **Deploy base model**:

Purpose	Model	Deployment name
Chat / answering	`gpt-4o-mini` (or `gpt-4.1-mini`)	`chat`
Embeddings	`text-embedding-3-small`	`embed`

The default "Global Standard" deployment type is fine. Write down the deployment names because your code uses them.

Step 3: Create Azure AI Search (the vector store)

1. In the Azure portal (portal.azure.com), go to **Create a resource** → **Azure AI Search**.
2. Use the same resource group and region as your project, and choose the **Free** pricing tier. It supports vector search.
3. When it's created, copy:
    - the **URL**, from the Overview page (`https://<name>.search.windows.net`)
    - the **Admin key**, from Settings → **Keys**

Step 4: Set up the local project

```
mkdir product-chatbot && cd product-chatbot
python3 -m venv .venv && source .venv/bin/activate
pip install openai azure-search-documents python-dotenv
```

Create `.env` and never commit it:

```
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_KEY=<key>
AZURE_OPENAI_API_VERSION=2024-10-21
CHAT_DEPLOYMENT=chat
EMBED_DEPLOYMENT=embed

AZURE_SEARCH_ENDPOINT=https://<your-search>.search.windows.net
AZURE_SEARCH_KEY=<admin-key>
AZURE_SEARCH_INDEX=products
```

Step 5: Create sample product data

Create `products.json`. You can replace it with your own catalog later.

```
[
  {"id": "1", "name": "AeroBook 14 Laptop", "category": "Laptops",
   "description": "14-inch laptop, Intel Core Ultra 7, 16GB RAM, 512GB SSD, 18-hour battery,
1.2kg. Price $1,099. 2-year warranty. Ports: 2x USB-C, 1x USB-A, HDMI."},
  {"id": "2", "name": "PulseBuds Pro", "category": "Audio",
   "description": "Wireless earbuds with active noise cancellation, 30h total battery with
case, IPX5 water resistance, Bluetooth 5.3. Price $179. 1-year warranty."},
  {"id": "3", "name": "HomeHub Smart Speaker", "category": "Smart Home",
   "description": "Voice-controlled speaker with Matter support, 360° sound, works with Alexa
and Google Assistant. Price $99. Colors: charcoal, sand."},
```

```
  {"id": "4", "name": "Return Policy", "category": "Policy",
   "description": "Products can be returned within 30 days of delivery in original packaging
for a full refund. Opened audio products have a 15-day window."}
]
```

Step 6: Ingest the data (`ingest.py`)

This script creates a search index that has a vector field, embeds each product, and uploads the results.

```
import json, os
from dotenv import load_dotenv
from openai import AzureOpenAI
from azure.core.credentials import AzureKeyCredential
from azure.search.documents import SearchClient
from azure.search.documents.indexes import SearchIndexClient
from azure.search.documents.indexes.models import (
    SearchIndex, SimpleField, SearchableField, SearchField, SearchFieldDataType,
    VectorSearch, HnswAlgorithmConfiguration, VectorSearchProfile,
)

load_dotenv()
aoai = AzureOpenAI(
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    api_version=os.environ["AZURE_OPENAI_API_VERSION"],
)
cred = AzureKeyCredential(os.environ["AZURE_SEARCH_KEY"])
endpoint, index_name = os.environ["AZURE_SEARCH_ENDPOINT"], os.environ["AZURE_SEARCH_INDEX"]

# 1. Define the index schema
fields = [
    SimpleField(name="id", type=SearchFieldDataType.String, key=True),
    SearchableField(name="name", type=SearchFieldDataType.String),
    SimpleField(name="category", type=SearchFieldDataType.String, filterable=True),
    SearchableField(name="content", type=SearchFieldDataType.String),
    SearchField(
        name="content_vector",
        type=SearchFieldDataType.Collection(SearchFieldDataType.Single),
        searchable=True,
        vector_search_dimensions=1536,          # text-embedding-3-small size
        vector_search_profile_name="vprofile",
    ),
]
vector_search = VectorSearch(
    algorithms=[HnswAlgorithmConfiguration(name="hnsw")],
    profiles=[VectorSearchProfile(name="vprofile", algorithm_configuration_name="hnsw")],
```

```
)
SearchIndexClient(endpoint, cred).create_or_update_index(
    SearchIndex(name=index_name, fields=fields, vector_search=vector_search)
)

# 2. Embed and upload
products = json.load(open("products.json"))
docs = []
for p in products:
    text = f"{p['name']} ({p['category']}): {p['description']}"
    vector = aoai.embeddings.create(model=os.environ["EMBED_DEPLOYMENT"],
input=text).data[0].embedding
    docs.append({"id": p["id"], "name": p["name"], "category": p["category"],
                 "content": text, "content_vector": vector})

SearchClient(endpoint, index_name, cred).upload_documents(docs)
print(f"Uploaded {len(docs)} documents to '{index_name}'")
```

Run it:

```
python ingest.py
```

Check: In the Azure portal, open your Search service → **Indexes** → `products` → **Search explorer**. Click Search and you should see your 4 documents.

> **Chunking:** These product descriptions are short, so each one is a single chunk. For long PDFs or manuals, split the text into pieces of about 500–1000 tokens with some overlap, and store each piece as its own document.

Step 7: Build the chat loop (`chat.py`)

```
import os
from dotenv import load_dotenv
from openai import AzureOpenAI
from azure.core.credentials import AzureKeyCredential
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery

load_dotenv()
aoai = AzureOpenAI(
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    api_version=os.environ["AZURE_OPENAI_API_VERSION"],
)
search = SearchClient(os.environ["AZURE_SEARCH_ENDPOINT"], os.environ["AZURE_SEARCH_INDEX"],
```

```
                    AzureKeyCredential(os.environ["AZURE_SEARCH_KEY"]))

SYSTEM_PROMPT = """You are a helpful product assistant for our store.
Answer ONLY using the product information in the <context> section.
If the answer is not in the context, say you don't know and suggest contacting support.
Cite the product name(s) you used in square brackets, e.g. [AeroBook 14 Laptop]."""

def retrieve(question: str, k: int = 3) -> str:
    vector = aoai.embeddings.create(model=os.environ["EMBED_DEPLOYMENT"],
input=question).data[0].embedding
    results = search.search(
        search_text=question,                                   # keyword part (hybrid)
        vector_queries=[VectorizedQuery(vector=vector, k_nearest_neighbors=k,
fields="content_vector")],
        select=["name", "content"],
        top=k,
    )
    return "\n\n".join(f"[{r['name']}]\n{r['content']}" for r in results)

history = [{"role": "system", "content": SYSTEM_PROMPT}]

while True:
    question = input("\nYou: ").strip()
    if question.lower() in {"exit", "quit"}:
        break
    context = retrieve(question)
    messages = history + [{"role": "user",
                           "content": f"<context>\n{context}\n</context>\n\nQuestion:
{question}"}]
    reply = aoai.chat.completions.create(
        model=os.environ["CHAT_DEPLOYMENT"], messages=messages, temperature=0.2
    ).choices[0].message.content
    print(f"\nBot: {reply}")
    # Keep the plain question and answer in history, not the bulky context
    history += [{"role": "user", "content": question}, {"role": "assistant", "content":
reply}]
```

Step 8: Test it

```
python chat.py
```

Try these questions:

- *"Which laptop do you have and how long does the battery last?"* → should mention the AeroBook and 18 hours

- *"Can I return earbuds I already opened?"* → should say 15 days
- *"Do you sell cameras?"* → should say it doesn't know. This shows the grounding works.

Learning exercise: Add `print(context)` inside the loop. Seeing exactly what the model receives is the best way to understand RAG.

Ideas for going further

1. **No-code comparison:** In the Foundry portal, open **Playgrounds** → **Chat** → **Add your data**, point it at your `products` index, and compare its answers with yours.
2. **Real documents:** Upload PDFs to Blob Storage and use the **Import and vectorize data** wizard in AI Search, which chunks and embeds them for you.
3. **Better retrieval:** Turn on **semantic ranker** in AI Search, or add a category filter, e.g. `filter="category eq 'Laptops'"`.
4. **Evaluation:** Use Foundry's **Evaluation** tab to score your answers for groundedness and relevance.

Part 2: Web app on Azure App Service

You'll put a Flask API and a simple HTML chat page around the RAG code from Part 1, then deploy it to App Service in the same resource group.

Architecture

```
Browser (index.html)
   │  POST /api/chat {question, history}
   ▼
Azure App Service (Flask + gunicorn)
   ├── embed question  → Foundry: text-embedding-3-small
   ├── hybrid search   → Azure AI Search (products index)
   └── generate answer → Foundry: gpt-4o-mini
```

The server is **stateless** because the browser keeps the chat history and sends it with each request. That works well on App Service, which can restart or scale your app at any time.

Step 1: Restructure the project

Keep the deployable code in its own folder. That way your `.venv` and `.env` don't get uploaded to Azure.

```
product-chatbot/
├── .venv/              ← local only
├── .env                ← local only (secrets)
├── ingest.py           ← from Part 1 (run locally, once)
├── products.json
└── webapp/             ← ONLY this folder gets deployed
    ├── app.py
    ├── rag.py
    ├── requirements.txt
    └── templates/
        └── index.html
```

```
cd product-chatbot
mkdir -p webapp/templates
pip install flask gunicorn
```

Step 2: `webapp/rag.py` (the RAG logic)

This is the logic from `chat.py`, moved into functions the web app can call.

```
import os
from dotenv import load_dotenv
from openai import AzureOpenAI
from azure.core.credentials import AzureKeyCredential
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery

load_dotenv()  # Locally: reads ../.env. On Azure: no file, uses App Settings instead.

aoai = AzureOpenAI(
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    api_version=os.environ["AZURE_OPENAI_API_VERSION"],
)
search = SearchClient(
    os.environ["AZURE_SEARCH_ENDPOINT"],
    os.environ["AZURE_SEARCH_INDEX"],
    AzureKeyCredential(os.environ["AZURE_SEARCH_KEY"]),
)

SYSTEM_PROMPT = """You are a helpful product assistant for our store.
Answer ONLY using the product information in the <context> section.
If the answer is not in the context, say you don't know and suggest contacting support.
Cite the product name(s) you used in square brackets, e.g. [AeroBook 14 Laptop]."""


def retrieve(question: str, k: int = 3) -> list[dict]:
    vector = aoai.embeddings.create(
        model=os.environ["EMBED_DEPLOYMENT"], input=question
    ).data[0].embedding
    results = search.search(
        search_text=question,
        vector_queries=[VectorizedQuery(vector=vector, k_nearest_neighbors=k,
fields="content_vector")],
        select=["name", "content"],
        top=k,
    )
    return [{"name": r["name"], "content": r["content"]} for r in results]


def answer(question: str, history: list[dict]) -> tuple[str, list[str]]:
    docs = retrieve(question)
    context = "\n\n".join(f"[{d['name']}]\n{d['content']}" for d in docs)
    messages = (
        [{"role": "system", "content": SYSTEM_PROMPT}]
        + history
```

```
        + [{"role": "user", "content": f"<context>\n{context}\n</context>\n\nQuestion:
{question}"}]
    )
    reply = aoai.chat.completions.create(
        model=os.environ["CHAT_DEPLOYMENT"], messages=messages, temperature=0.2
    ).choices[0].message.content
    return reply, [d["name"] for d in docs]
```

Step 3: `webapp/app.py` (the Flask API)

```
from flask import Flask, jsonify, render_template, request
from rag import answer

app = Flask(__name__)


@app.get("/")
def index():
    return render_template("index.html")


@app.post("/api/chat")
def chat():
    data = request.get_json(silent=True) or {}
    question = (data.get("question") or "").strip()
    if not question:
        return jsonify(error="Question is required"), 400
    if len(question) > 1000:
        return jsonify(error="Question is too long"), 400

    # Only accept user/assistant turns from the browser, so a client can't inject a system
prompt
    history = [
        {"role": m["role"], "content": str(m["content"])}
        for m in data.get("history", [])
        if isinstance(m, dict) and m.get("role") in ("user", "assistant")
    ][-10:]

    try:
        reply, sources = answer(question, history)
    except Exception as e:
        app.logger.exception("Chat failed")
        return jsonify(error="Something went wrong. Please try again."), 500
    return jsonify(reply=reply, sources=sources)
```

```
@app.get("/health")
def health():
    return "ok"
```

Step 4: `webapp/templates/index.html` (the chat UI)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Product Assistant</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 720px; margin: 0 auto; padding:
16px; background: #f6f7f9; }
    h1 { font-size: 1.4rem; }
    #chat { background: #fff; border-radius: 12px; padding: 16px; height: 65vh; overflow-y:
auto; box-shadow: 0 1px 4px #0001; }
    .msg { margin: 10px 0; padding: 10px 14px; border-radius: 10px; max-width: 85%; white-
space: pre-wrap; line-height: 1.4; }
    .user { background: #0b6cff; color: #fff; margin-left: auto; }
    .bot  { background: #eef0f3; }
    .sources { font-size: .8rem; color: #666; margin-top: 6px; }
    form { display: flex; gap: 8px; margin-top: 12px; }
    input { flex: 1; padding: 12px; border-radius: 8px; border: 1px solid #ccc; font-size:
1rem; }
    button { padding: 12px 18px; border: 0; border-radius: 8px; background: #0b6cff; color:
#fff; font-size: 1rem; cursor: pointer; }
    button:disabled { opacity: .5; }
  </style>
</head>
<body>
  <h1>👜 Product Assistant</h1>
  <div id="chat"><div class="msg bot">Hi! Ask me about our products, prices, or return policy.
</div></div>
  <form id="form">
    <input id="q" placeholder="e.g. How long does the AeroBook battery last?"
autocomplete="off" required>
    <button id="send">Send</button>
  </form>

<script>
  const chat = document.getElementById("chat");
  const form = document.getElementById("form");
```

```
  const input = document.getElementById("q");
  const send = document.getElementById("send");
  const history = [];

  function add(text, cls, sources) {
    const div = document.createElement("div");
    div.className = "msg " + cls;
    div.textContent = text;                      // textContent, not innerHTML, avoids XSS
    if (sources?.length) {
      const s = document.createElement("div");
      s.className = "sources";
      s.textContent = "Sources: " + sources.join(", ");
      div.appendChild(s);
    }
    chat.appendChild(div);
    chat.scrollTop = chat.scrollHeight;
    return div;
  }

  form.addEventListener("submit", async (e) => {
    e.preventDefault();
    const question = input.value.trim();
    if (!question) return;
    add(question, "user");
    input.value = "";
    send.disabled = true;
    const thinking = add("Thinking…", "bot");

    try {
      const res = await fetch("/api/chat", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ question, history }),
      });
      const data = await res.json();
      thinking.remove();
      if (!res.ok) throw new Error(data.error || "Request failed");
      add(data.reply, "bot", data.sources);
      history.push({ role: "user", content: question }, { role: "assistant", content:
data.reply });
    } catch (err) {
      thinking.remove();
      add("⚠️ " + err.message, "bot");
    } finally {
      send.disabled = false;
      input.focus();
    }
```

```
  });
</script>
</body>
</html>
```

Step 5: `webapp/requirements.txt`

App Service reads this file and installs the packages during deployment.

```
flask
gunicorn
openai
azure-search-documents
python-dotenv
```

Step 6: Test locally

```
cd webapp
flask --app app run --debug
```

Open http://127.0.0.1:5000 and ask the same test questions from Part 1. **Only deploy once it works locally.** Debugging in the cloud is much slower.

Step 7: Deploy to Azure App Service

7a. Log in and set variables

```
az login

# Change these. APP_NAME must be globally unique (it becomes <name>.azurewebsites.net)
RG=rg-product-chatbot
LOCATION=eastus2                # use the same region as your other resources
APP_NAME=product-chatbot-$RANDOM
echo $APP_NAME                  # write this down
```

7b. Create and deploy the app in one command

Run this from **inside the `webapp/` folder**:

```
cd webapp
az webapp up \
  --name $APP_NAME \
  --resource-group $RG \
```

```
  --location $LOCATION \
  --runtime "PYTHON:3.12" \
  --sku B1
```

This command:

- creates an **App Service Plan** (Linux, B1, about $13/month, billed hourly) and a **Web App**
- zips the current folder and uploads it
- runs `pip install -r requirements.txt` on Azure

> To spend less, you can use `--sku F1` (Free). It is slow and limited to 60 CPU minutes a day, but that's enough for a demo.

7c. Set the startup command

```
az webapp config set \
  --resource-group $RG --name $APP_NAME \
  --startup-file "gunicorn --bind=0.0.0.0 --timeout 120 app:app"
```

7d. Add your settings (environment variables)

Your `.env` file is **not** deployed. Copy its values into App Settings instead:

```
az webapp config appsettings set \
  --resource-group $RG --name $APP_NAME \
  --settings \
    AZURE_OPENAI_ENDPOINT="https://<your-resource>.openai.azure.com/" \
    AZURE_OPENAI_API_KEY="<key>" \
    AZURE_OPENAI_API_VERSION="2024-10-21" \
    CHAT_DEPLOYMENT="chat" \
    EMBED_DEPLOYMENT="embed" \
    AZURE_SEARCH_ENDPOINT="https://<your-search>.search.windows.net" \
    AZURE_SEARCH_KEY="<admin-key>" \
    AZURE_SEARCH_INDEX="products"
```

Changing settings restarts the app automatically.

> **Tip:** For a web app, use a **query key** instead of the admin key. In the Search service, go to **Keys** → **Manage query keys** → **Add**. A query key can only read, so a leaked key can't modify your index. (Part 3 removes keys entirely.)

7e. Open the app

```
az webapp browse --resource-group $RG --name $APP_NAME
```

Or go to `https://<APP_NAME>.azurewebsites.net`. The first load can take 30–60 seconds while the container starts.

7f. Redeploy after code changes

From inside `webapp/`, run the same command again:

```
az webapp up --name $APP_NAME --resource-group $RG
```

Step 8: Troubleshooting

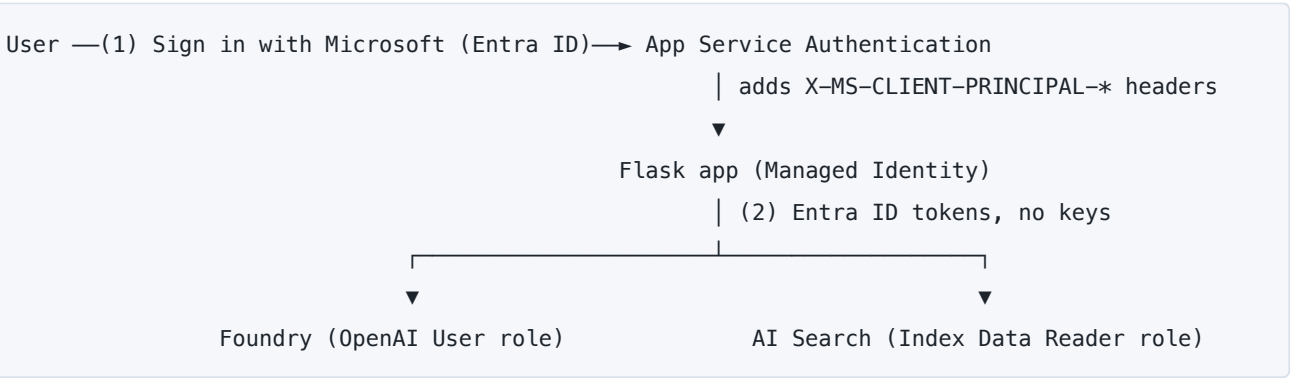
Symptom	What to check
"Application Error" page	`az webapp log tail -g $RG -n $APP_NAME` shows the Python traceback
`KeyError: 'AZURE_OPENAI_ENDPOINT'`	An App Setting is missing or misspelled (step 7d)
`ModuleNotFoundError`	`requirements.txt` wasn't in the folder you deployed from
401 from OpenAI or Search	Wrong key, or a stray space or quote when you pasted it
404 `DeploymentNotFound`	`CHAT_DEPLOYMENT` / `EMBED_DEPLOYMENT` must match your deployment names in Foundry
Page is very slow at first	Cold start. That's normal on B1/F1.

You can also view logs in the portal: Web App → **Log stream**.

Part 3: Microsoft Entra ID authentication

This part uses Entra ID in two places:

Layer	What it protects	How
User → web app	Only people in your Entra tenant can open the chatbot	App Service Authentication ("Easy Auth"), with no login code to write
Web app → Azure services	The app calls Foundry and AI Search **without API keys**	Managed Identity + `DefaultAzureCredential` + Azure RBAC roles



What changes from Part 2:

- `ingest.py` and `rag.py` drop API keys and use `DefaultAzureCredential`.
- `requirements.txt` adds `azure-identity`.
- `.env` and App Settings no longer contain any keys.
- New steps: enable RBAC on Search, assign roles, turn on Managed Identity, and configure Easy Auth.
- `app.py` and `index.html` show who is signed in and add a sign-out link.

Prerequisite: You need the **Owner** or **User Access Administrator** role on the resource group to assign roles. If you created the subscription yourself, you already have Owner.

Step 1: Set the shell variables used throughout

```
az login                                  # DefaultAzureCredential uses this login locally

RG=rg-product-chatbot
LOCATION=eastus2
AI_NAME=<your-foundry-resource-name>      # the <name> in https://<name>.openai.azure.com
SEARCH_NAME=<your-search-service-name>    # the <name> in https://<name>.search.windows.net
APP_NAME=<your-web-app-name>              # from Part 2, or a new unique name

AI_ID=$(az cognitiveservices account show -n $AI_NAME -g $RG --query id -o tsv)
```

```
SEARCH_ID=$(az search service show -n $SEARCH_NAME -g $RG --query id -o tsv)
ME=$(az ad signed-in-user show --query id -o tsv)
```

If you can't find the Foundry resource name, run `az cognitiveservices account list -g $RG -o table`.

Step 2: Enable role-based access on AI Search

AI Search only accepts API keys by default. This setting lets it accept **both** Entra ID tokens and keys:

```
az search service update -n $SEARCH_NAME -g $RG \
  --auth-options aadOrApiKey \
  --aad-auth-failure-mode http401WithBearerChallenge
```

(In the portal: Search service → **Settings** → **Keys** → API access control → **Both**.)

Step 3: Give yourself roles (for local dev and ingestion)

Locally, `DefaultAzureCredential` signs in as **you**, through `az login`. So your user account needs these roles:

```
# Call the embedding and chat models
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Cognitive Services OpenAI User" --scope $AI_ID

# Create the index (ingest.py)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Search Service Contributor" --scope $SEARCH_ID

# Upload and read documents
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Search Index Data Contributor" --scope $SEARCH_ID
```

⏳ **Role assignments take 5–10 minutes to take effect.** A 401 or 403 right after this step is normal.

Step 4: Remove keys from `.env`

```
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_VERSION=2024-10-21
CHAT_DEPLOYMENT=chat
EMBED_DEPLOYMENT=embed
```

```
AZURE_SEARCH_ENDPOINT=https://<your-search>.search.windows.net
AZURE_SEARCH_INDEX=products
```

Delete `AZURE_OPENAI_API_KEY` and `AZURE_SEARCH_KEY`. The code doesn't use them anymore.

Step 5: Update the code to be keyless

`webapp/requirements.txt`

```
flask
gunicorn
openai
azure-search-documents
azure-identity
python-dotenv
```

Install `azure-identity` locally as well: `pip install azure-identity`

`webapp/rag.py`: replace only the client setup at the top

```
import os
from dotenv import load_dotenv
from openai import AzureOpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery

load_dotenv()

# Locally: uses your `az login` account. On App Service: uses the Managed Identity.
credential = DefaultAzureCredential()

aoai = AzureOpenAI(
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    azure_ad_token_provider=get_bearer_token_provider(
        credential, "https://cognitiveservices.azure.com/.default"
    ),
    api_version=os.environ["AZURE_OPENAI_API_VERSION"],
)
search = SearchClient(
    os.environ["AZURE_SEARCH_ENDPOINT"],
    os.environ["AZURE_SEARCH_INDEX"],
    credential,
)
```

```
# ... SYSTEM_PROMPT, retrieve() and answer() stay exactly the same
```

`ingest.py` : same idea

Replace the imports and the client setup, and pass `credential` wherever `cred` was used before:

```
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
# (remove: from azure.core.credentials import AzureKeyCredential)

credential = DefaultAzureCredential()
aoai = AzureOpenAI(
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    azure_ad_token_provider=get_bearer_token_provider(
        credential, "https://cognitiveservices.azure.com/.default"
    ),
    api_version=os.environ["AZURE_OPENAI_API_VERSION"],
)
endpoint, index_name = os.environ["AZURE_SEARCH_ENDPOINT"], os.environ["AZURE_SEARCH_INDEX"]

# ...index definition unchanged...
SearchIndexClient(endpoint, credential).create_or_update_index(...)
# ...
SearchClient(endpoint, index_name, credential).upload_documents(docs)
```

`webapp/app.py` : show who is signed in

Easy Auth adds the user's identity to every request as HTTP headers. Add a helper and pass the user to the template:

```
from flask import Flask, jsonify, render_template, request
from rag import answer

app = Flask(__name__)


def current_user() -> str:
    # Injected by App Service Authentication (Easy Auth). Absent when running locally.
    return request.headers.get("X-MS-CLIENT-PRINCIPAL-NAME", "local-dev")


@app.get("/")
def index():
    return render_template("index.html", user=current_user())
```

```
@app.post("/api/chat")
def chat():
    data = request.get_json(silent=True) or {}
    question = (data.get("question") or "").strip()
    if not question:
        return jsonify(error="Question is required"), 400
    if len(question) > 1000:
        return jsonify(error="Question is too long"), 400

    history = [
        {"role": m["role"], "content": str(m["content"])}
        for m in data.get("history", [])
        if isinstance(m, dict) and m.get("role") in ("user", "assistant")
    ][-10:]

    app.logger.info("Question from %s", current_user())
    try:
        reply, sources = answer(question, history)
    except Exception:
        app.logger.exception("Chat failed")
        return jsonify(error="Something went wrong. If you were idle, refresh the page to sign
in again."), 500
    return jsonify(reply=reply, sources=sources)


@app.get("/health")
def health():
    return "ok"
```

> **Why trusting this header is safe:** On App Service, all traffic passes through the Easy Auth layer. It removes any client-sent `X-MS-*` headers and adds the real ones. Locally, no one sets the header, so you see `local-dev`.

`webapp/templates/index.html` : add a header bar

Replace the `<h1>` line with this:

```
<header>
  <h1>🛍️ Product Assistant</h1>
  <div class="who">{{ user }} · <a href="/.auth/logout">Sign out</a></div>
</header>
```

Add these rules to the `<style>` block:

```
header { display: flex; justify-content: space-between; align-items: center; gap: 8px; flex-
wrap: wrap; }
```

```
.who { font-size: .85rem; color: #555; }
```

Jinja escapes `{{ user }}` automatically. The sign-out link returns a 404 when you run locally, which is expected.

Step 6: Test locally

```
python ingest.py              # proves your user roles work for Search + embeddings
cd webapp && flask --app app run --debug
```

If you get `DefaultAzureCredential failed to retrieve a token`, run `az login` again. If your account has several tenants, use `az login --tenant <tenant-id>`.

Step 7: Deploy and turn on Managed Identity

7a. Deploy the code

Run this from `webapp/`, the same way as in Part 2. If the app already exists, this just redeploys it.

```
cd webapp
az webapp up --name $APP_NAME --resource-group $RG --location $LOCATION \
  --runtime "PYTHON:3.12" --sku B1

az webapp config set -g $RG -n $APP_NAME \
  --startup-file "gunicorn --bind=0.0.0.0 --timeout 120 app:app"
```

7b. Enable a system-assigned Managed Identity

```
APP_PRINCIPAL=$(az webapp identity assign -g $RG -n $APP_NAME --query principalId -o tsv)
echo $APP_PRINCIPAL
```

7c. Grant the web app only the roles it needs

The app only needs to read, so it gets fewer permissions than you:

```
az role assignment create --assignee-object-id $APP_PRINCIPAL --assignee-principal-type
ServicePrincipal \
  --role "Cognitive Services OpenAI User" --scope $AI_ID

az role assignment create --assignee-object-id $APP_PRINCIPAL --assignee-principal-type
ServicePrincipal \
  --role "Search Index Data Reader" --scope $SEARCH_ID
```

7d. Set App Settings without keys

```
az webapp config appsettings set -g $RG -n $APP_NAME --settings \
  AZURE_OPENAI_ENDPOINT="https://$AI_NAME.openai.azure.com/" \
  AZURE_OPENAI_API_VERSION="2024-10-21" \
  CHAT_DEPLOYMENT="chat" \
  EMBED_DEPLOYMENT="embed" \
  AZURE_SEARCH_ENDPOINT="https://$SEARCH_NAME.search.windows.net" \
  AZURE_SEARCH_INDEX="products"

# If you followed Part 2, remove the old keys:
az webapp config appsettings delete -g $RG -n $APP_NAME \
  --setting-names AZURE_OPENAI_API_KEY AZURE_SEARCH_KEY
```

Wait about 5–10 minutes for the roles to take effect, then open the app and test a question.

Step 8: Require users to sign in (Easy Auth)

The portal is the simplest way to do this, because it creates the app registration and client secret for you.

1. Portal → **App Services** → your app → **Settings** → **Authentication** → **Add identity provider**.
2. **Identity provider:** Microsoft
3. **Tenant type:** Workforce configuration (current tenant)
4. **App registration type:** Create new app registration
    - **Name:** leave the default (your app name)
    - **Supported account types:** *Current tenant – Single tenant*
    - **Client secret expiration:** 180 days is fine
5. **Additional checks:** leave the defaults. That means requests are allowed only from this application, from any identity, and only from the issuer tenant.
6. **App Service authentication settings:**
    - **Restrict access:** Require authentication
    - **Unauthenticated requests:** HTTP 302 Found redirect (recommended for websites)
    - **Token store:** ✅ checked
7. **Permissions:** keep `User.Read`, then click **Add**.

Test it: Open `https://<APP_NAME>.azurewebsites.net` in a **private/incognito window**. You should be redirected to the Microsoft sign-in page. The first time, you'll see a consent prompt. After signing in, the header shows your email and a **Sign out** link.

> **What just happened:** Azure created an **App Registration** in Entra ID, stored its secret in the App Setting `MICROSOFT_PROVIDER_AUTHENTICATION_SECRET`, and turned on an auth layer in front of your Flask app. Your code didn't change at all.

Step 9 (recommended): Allow only specific users

By default, **anyone in your tenant** can sign in. To limit access to named people:

1. Portal → **Microsoft Entra ID** → **Enterprise applications**. Remove the filter so it shows *All applications*, then open your app.
2. **Properties** → **Assignment required?** → **Yes** → Save.
3. **Users and groups** → **Add user/group** → add yourself and anyone else who should have access.
    - Assigning *groups* needs Entra ID P1. On the free tier, assign individual users.
    - To let someone from outside your tenant in, first invite them: Entra ID → **Users** → **Invite external user**.

Anyone else who signs in gets error `AADSTS50105`, meaning they're not assigned to the app.

Step 10 (optional): Turn off API keys completely

This proves the app is truly keyless. After this, any code that still uses a key fails.

```
az search service update -n $SEARCH_NAME -g $RG --disable-local-auth true
az resource update --ids $AI_ID --set properties.disableLocalAuth=true
```

Your app and `ingest.py` should still work, because they use Entra ID tokens now.

Troubleshooting

Symptom	Fix
`401` / `403 PermissionDenied` right after assigning roles	Wait 5–10 minutes for the roles to propagate, then restart: `az webapp restart -g $RG -n $APP_NAME`
Search returns `403 Forbidden`	RBAC isn't enabled on Search (Step 2), or a role is missing (Step 3 or 7c)
`AuthorizationFailed` when running `az role assignment create`	You need Owner or User Access Administrator on the resource group
`DefaultAzureCredential failed` locally	`az login`, or `az login --tenant <id>`
`ManagedIdentityCredential` errors on Azure	Step 7b wasn't run, or the roles in 7c went to a different principal ID
`AADSTS50105` at sign-in	Assignment is required (Step 9) but that user isn't assigned
Chat fails after being idle a long time	The sign-in session expired. Refresh the page.
See all errors	`az webapp log tail -g $RG -n $APP_NAME`

To check role assignments:

```
az role assignment list --scope $SEARCH_ID -o table
az role assignment list --scope $AI_ID -o table
```

Cleanup: delete everything

Entra ID objects belong to the **tenant**, not the resource group, so the app registration needs a separate delete.

```
# 1. Delete the Entra app registration created by Easy Auth (do this first, while you still
know the name)
APP_REG_ID=$(az ad app list --display-name $APP_NAME --query "[0].appId" -o tsv)
az ad app delete --id $APP_REG_ID

# 2. Delete all Azure resources (Web App + its Managed Identity, Plan, Search, Foundry)
az group delete --name $RG --yes --no-wait

# 3. Purge the soft-deleted Foundry/AI resource
az cognitiveservices account list-deleted -o table
az cognitiveservices account purge --name $AI_NAME --resource-group $RG --location $LOCATION

# 4. Verify
az group exists --name $RG                                  # false
az ad app list --display-name $APP_NAME -o table            # empty
```

The system-assigned Managed Identity and its role assignments are deleted along with the web app and the resource group.

> **Why step 3:** Azure keeps deleted Foundry/AI Services resources in a "soft-deleted" state for 48 hours. During that time the name stays reserved. Purging removes it fully.

Check **Cost Management** in the Azure portal a day later to confirm nothing is still billing.

Final checklist

- □ Part 1: Foundry project, two model deployments, AI Search, `ingest.py`, and `chat.py` working
- □ Part 2: Flask app works locally and is deployed to App Service
- □ Part 3: RBAC on Search, roles assigned, Managed Identity on, keys removed
- □ Part 3: Easy Auth sign-in works in a private window
- □ Cleanup: app registration deleted, resource group deleted, AI resource purged